■ OFO-DBMS: Project Status & Evaluation Report

Online Food Ordering & Delivery System — Dual-Database Relational Architecture

Status: Production-Ready Prototype (v1.0.0)	**Tech Stack:** Python 3.10+, Django 5, SQLite, Bootstrap 5
Test Suite: 28/28 Unit & Integration Tests Passing (100%)	**Architecture:** Dual-Database with Custom Router & Atomic Tx
Date Generated: October 2026	**Repository:** c:\Users\Mahendra\OFO-DBMS

1. Executive Summary

OFO-DBMS is a comprehensive, production-inspired Online Food Ordering platform built to showcase advanced relational database management principles in Python and Django. Unlike basic CRUD demonstrations, the system implements a decoupled dual-database architecture separating business domain entities from framework internals, enforces multi-table ACID transactions, incorporates multi-parameter search/filtering algorithms, and features both customer-facing storefronts and an executive restaurant administration console.

Current State: The codebase is entirely healthy, completely migrated, clean in version control (branch main), and exhibits a **100% test pass rate** across 28 rigorous automated tests covering customer lifecycle, session carts, checkout validation, and kitchen order management.

2. What Has Been Done Till Now (Delivered Features)

A. Dual-Database Architecture & Relational Schema

- **Dual SQLite Engine:** food_ordering.db houses business models (Orders, Menus, Payments, Deliveries), while django_database.db isolates framework tables (Sessions, Auth, ContentTypes, Admin logs).
- **Custom Router:** DjangoFrameworkRouter seamlessly routes reads, writes, and migrations based on app labels.
- **10 Entity Relational Model:** CustomerProfile (1:1), Restaurant to MenuItems (1:N), Category to MenuItems (1:N), Order to OrderItems (1:N), Order to Payment (1:1), Order to Delivery (1:1), DeliveryPartner to Delivery (1:N), Reviews (1:N).
- **Automated Seeder:** Custom Django command python manage.py seed_db generates 6 categories, 5 restaurants, 15 dishes, 3 delivery drivers, sample orders, reviews, and test user credentials.

B. Customer Storefront & Menu Discovery

- **Interactive Storefront:** Grid layout built with Bootstrap 5 cards displaying dish photos, pricing, prep times, veg/non-veg status, and restaurant details.
- **Multi-Criteria Discovery:** Dynamic search (dish, restaurant, description), category quick-carousel, veg/non-veg toggle, and 4 sorting modes (Price Asc/Desc, Prep Time, Rating).
- **Reviews & Ratings:** Verified customer review submissions with automatic dynamic recalculation of restaurant aggregate ratings.

C. Cart, Checkout & Order Lifecycle

- **Session-Based Cart:** Persistent session cart supporting guests and registered users, item increment/decrement, and a navbar live badge counter via context processor.
- **Atomic Checkout Workflow:** Wrapped in transaction.atomic() to guarantee synchronized creation of Order, OrderItems, Payment, and initial Delivery partner allocation.
- **Validation Engine:** Strict regex verification for phone numbers, address length thresholds, and payment mode checks.
- **Visual 4-Stage Tracker:** Step progress tracker: *Order Placed* → *Kitchen Preparing* → *Out for Delivery* → *Delivered*.
- **Printable Tax Invoice:** Dedicated receipt view with itemized billing, unit rates, restaurant attribution, and payment status.

D. Restaurant Operations & Admin Dashboard

- **Operational KPIs:** Real-time metric cards for Total Revenue, Total Orders, Active Kitchen Queue, Completed Deliveries, Dish Inventory, and Customer Base.
- **Live Kitchen Queue:** 1-click status progression (Preparing, Dispatched, Delivered, Cancelled) and driver assignment dropdown.
- **Stock Availability Switch:** Real-time toggle between *In Stock* and *Sold Out* per dish.
- **Business Analytics:** Aggregated reports for Top 5 Best-Selling Dishes, Payment Mode Distributions, and Category-Wise Revenue.

3. Relational Schema & Entity Relationship Summary

The following table highlights the core database entities, their relationship mappings, and key functional constraints:

Entity	Relationship	Database	Key Attributes / Responsibilities
CustomerProfile	1-to-1 with User	food_ordering.db	Phone number, delivery address (cross-DB constraint handled gracefully)
Category	1-to-Many MenuItems	food_ordering.db	Category name, Bootstrap icon class, category description
Restaurant	1-to-Many MenuItems	food_ordering.db	Restaurant name, physical address, aggregated rating (1.0-5.0), cover image
MenuItem	Belongs to Restaurant & Cat	food_ordering.db	Dish title, price, is_veg flag, prep time (mins), availability toggle
Order	1-to-Many OrderItems	food_ordering.db	Customer reference, timestamp, order status, total amount, special instructions
OrderItem	Links Order to MenuItem	food_ordering.db	Order ID, MenuItem ID, quantity ordered (PROTECT on dish deletion)
Payment	1-to-1 with Order	food_ordering.db	Mode (COD/UPI/Card), Status (Pending/Paid), transaction amount
DeliveryPartner	1-to-Many Deliveries	food_ordering.db	Driver name, contact phone number, active delivery assignments
Delivery	1-to-1 with Order	food_ordering.db	Order ID, driver partner FK, delivery status, timestamp on delivery
Review	Links Customer to Restaurant	food_ordering.db	1-5 star rating, customer comment, timestamp, menu item association

4. What Can Still Be Done (Gaps & Enhancement Opportunities)

To take this project from an academic / prototype milestone to a true commercial-grade SaaS delivery system, the following modules represent the highest-value opportunities:

Enhancement Area	Current Implementation	Proposed Enhancement & Tech
Payment Gateway Integration	Simulated via form select (marks Paid immediately for Card/UPI).	Integrate **Razorpay, Stripe, or PayPal** with webhook validation, checkout modal, and automated refunds on cancellation.
Real-Time Tracking & WebSockets	Status tracking requires manual browser page refresh.	Implement **Django Channels** or **Server-Sent Events (SSE)** for real-time status stepper updates and kitchen audio chime alerts.
Single-Restaurant Cart Rule	Cart allows mixing dishes from multiple different restaurants.	Enforce single-vendor cart constraint (prompting user to clear cart when switching restaurants) or split multi-vendor deliveries.
Discounts, Promo Codes & Taxes	Delivery fee fixed at 0.00; no coupon discount system.	Add Coupon model (flat/percentage off, min order value), dynamic distance delivery fee, and automated GST/tax calculation.

Live Map & Driver GPS	Driver name and phone rendered as plain text.	Embed **Google Maps API** or **Leaflet / OpenStreetMap** with real-time driver coordinates and route visualization.
Automated Notifications	In-app alert banners only.	Automated transactional emails (Order confirmation PDF invoice via SendGrid/SMTP) and SMS/WhatsApp updates via Twilio.
REST APIs & Decoupled Clients	Server-side HTML rendering with Django templates.	Expose **Django REST Framework (DRF)** endpoints to support mobile clients (Flutter / React Native) and external integrations.
Production Database Migration	Dual SQLite database files.	Upgrade to **PostgreSQL** or **MySQL** in Docker/Cloud for high-concurrency write transactions and production scaling.

5. Recommended Implementation Roadmap

Phase	Target Features	Estimated Effort & Impact
Phase 1 (Commercial Essentials)	• Single-restaurant cart validation • Promo codes & coupon engine • Dynamic delivery charges & tax breakdown	**High Impact / Low-Medium Effort** Immediate improvement to business rules and checkout realism.
Phase 2 (Payments & Alerts)	• Razorpay or Stripe live checkout integration • Webhook verification & refund management • Transactional Email invoices (SendGrid/SMTP)	**High Impact / Medium Effort** Enables true cashless payments and customer communication.
Phase 3 (Real-Time & Scale)	• Live WebSocket updates with Django Channels • Interactive driver tracking with Google Maps / Leaflet • Django REST Framework (DRF) APIs for mobile apps • Migration from SQLite to PostgreSQL	**Strategic / Advanced Effort** Transforms the system into a modern, production-grade distributed architecture.